Build an AI Internship Mentor Chatbot: Beginner Guide (Kiro + Gemini + Supabase)

Oct 28, 2026 · @tawfiq

What you are building

You will build a web app where interns log in, follow a 48-week Data Analytics roadmap, chat with a Gemini-powered mentor, tick off weekly tasks, rate the app, and see "Congratulations! You have successfully completed your internship!" at the end. Plan on 1 to 2 days, mostly pasting prompts into Kiro and checking what it builds.

Layer	Tool	What it does
Editor	Kiro	AI code editor that writes the code from your prompts
Frontend + backend	Next.js (React, TypeScript, Tailwind)	Pages and the secure `/api/chat` route, all in one project
Database + login	Supabase	Postgres tables, email login, per-user data protection
AI	Gemini API (Google AI Studio)	Explains concepts, gives hints and coding exercises
Code hosting	GitHub	Stores your code (required by the assignment)
Deployment	Vercel	Gives you the public shareable link

What the finished app does

- Email sign up and login
- 12-month roadmap: Excel, SQL, Python, Power BI, visualization, statistics, final project
- One task per week (48 tasks), with a "Mark complete" button and a progress bar
- AI mentor chat that knows your current task, with saved chat history
- Feedback form: 1 to 5 stars plus a comment, saved in the database
- Congratulations screen once every task is complete

How one chat message travels

1. The intern types a question in the chat page.

2. The browser sends it to your own `/api/chat` route.
3. The route checks the intern is logged in through Supabase.
4. The route sends the question, recent history and the current task to Gemini.
5. The route saves the question and the answer in the `chat_messages` table.
6. The answer appears in the chat window.

The Gemini key stays on the server. If it were in frontend code, anyone could copy it from the browser.

Step 1: Set up accounts and tools

Create every account below first, using the same email or "Sign in with GitHub" where offered. This takes about 20 minutes.

Tool	Where	Why you need it
Kiro	kiro.dev	The editor. Download, install, sign in
Node.js (LTS version)	nodejs.org	Runs the Next.js app on your laptop
Git	git-scm.com	Uploads your code to GitHub
GitHub account	github.com	Stores the code
Supabase account	supabase.com	Database and login
Google account	aistudio.google.com	Gemini API key
Vercel account	vercel.com	Hosting. Sign up with GitHub

Check that Node and Git installed. Open a terminal (in Kiro: Terminal menu, New Terminal) and run:

```
node -v
git --version
```

Both should print a version number. If you get "command not found", reinstall and restart Kiro.

- [ ] Kiro installed and signed in
- [ ] Node.js and Git installed
- [ ] GitHub, Supabase, Vercel accounts created

Step 2: Get your Gemini API key

The API key is a secret password that lets your app talk to Gemini. Google AI Studio gives one for free.

1. Open aistudio.google.com and sign in with your Google account.
2. Click **Get API key** (left sidebar or top right).
3. Click **Create API key**. If it asks for a project, pick **Create API key in new project**.
4. Copy the key. It looks like `AIza...` followed by many characters.
5. Paste it into a private note for now. You will put it in `.env.local` in step 6.
6. In AI Studio, open the model list and note the name of a current fast model, such as `gemini-2.5-flash`. Model names change over time, so your app will read the name from an environment variable instead of hardcoding it.

Rules for the key

- Never paste it into chat prompts, screenshots, GitHub, or frontend code.
- If it leaks, go back to AI Studio, delete it and create a new one.
- The free tier has per-minute and per-day limits. If you see a "429" error while testing, wait a minute and try again.
- Free-tier prompts may be used by Google to improve its products, so never send real personal data in your test chats.

Step 3: Create the database in Supabase

Create the project

1. Go to supabase.com, click **New project**, and name it `internship-mentor`.
2. Set a database password and save it in your private note. Pick the region closest to you (for India, Mumbai or Singapore).
3. Wait about 2 minutes until the project is ready.
4. Open **Project Settings, then API** (some versions label it **API Keys**). Copy the **Project URL** and the **anon / publishable key**. You need both in step 6.
5. Open **Authentication, then Providers, then Email** and turn **Confirm email** off. This lets you test signup without waiting for emails.

Create the tables

Open **SQL Editor, then New query**, paste the block below, and click **Run**. It creates five tables: `profiles`, `tasks`, `user_progress`, `chat_messages`, `feedback`.

```
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text,
  created_at timestamptz default now()
);

create table public.tasks (
  id serial primary key,
  week_number int not null,
  month_number int not null,
  skill text not null,
  title text not null,
  description text not null,
  is_final_project boolean default false
);

create table public.user_progress (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  task_id int not null references public.tasks(id) on delete cascade,
  completed_at timestamptz default now(),
  unique (user_id, task_id)
);

create table public.chat_messages (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  role text not null check (role in ('user', 'assistant')),
  content text not null,
  created_at timestamptz default now()
);

create table public.feedback (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id) on delete set null,
  rating int not null check (rating between 1 and 5),
  comment text,
  created_at timestamptz default now()
);
```

Protect each intern's data

Run this second block in a new query. Row Level Security means an intern can only read and write their own rows. It also creates a profile row automatically at signup.

```
alter table public.profiles enable row level security;
alter table public.tasks enable row level security;
alter table public.user_progress enable row level security;
alter table public.chat_messages enable row level security;
alter table public.feedback enable row level security;

create policy "own profile" on public.profiles
  for all using (auth.uid() = id) with check (auth.uid() = id);

create policy "tasks readable by logged-in users" on public.tasks
  for select using (auth.role() = 'authenticated');

create policy "own progress" on public.user_progress
  for all using (auth.uid() = user_id) with check (auth.uid() = user_id);

create policy "own chats" on public.chat_messages
  for all using (auth.uid() = user_id) with check (auth.uid() = user_id);

create policy "own feedback" on public.feedback
  for all using (auth.uid() = user_id) with check (auth.uid() = user_id);

create function public.handle_new_user()
returns trigger
language plpgsql
security definer set search_path = public
as $$
begin
  insert into public.profiles (id, full_name)
  values (new.id, new.raw_user_meta_data->>'full_name');
  return new;
end;
$$;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

Roadmap plan (the data Kiro will load into `tasks`)

The plan has 12 months with 4 weekly tasks each, so 48 tasks. In step 5, Kiro writes the `seed.sql` file for this.

Month	Skill	Focus
1	Excel	Formulas, cleaning, sorting, filtering

Month	Skill	Focus
2	Excel	Pivot tables, charts, a first dashboard
3	SQL	SELECT, WHERE, GROUP BY, aggregates
4	SQL	Joins, subqueries, window functions
5	Python	Syntax, loops, functions, files
6	Python	pandas and NumPy for analysis
7	Data visualization	matplotlib, seaborn, chart design
8	Power BI	Importing data, cleaning, reports
9	Power BI	DAX measures, interactive dashboards
10	Statistics	Mean, variance, distributions, hypothesis tests
11	Statistics + storytelling	Regression basics, presenting insights
12	Final project	End-to-end analysis: data, SQL or Python, dashboard, report

- [ ] Supabase project created
- [ ] Both SQL blocks ran without errors
- [ ] Project URL and anon key saved privately
- [ ] Confirm email turned off

Step 4: Set up Kiro and generate the project scaffold

1. Create an empty folder on your computer named `internship-mentor`.
2. In Kiro, choose **File, then Open Folder**, and select that folder.
3. Open the Kiro chat panel. You will see two ways to work: **Vibe** (chat and it builds, fastest) and **Spec** (it first writes requirements, design and task files, then builds). Use **Vibe** for speed. Use **Spec** for the master prompt below if you want those documents to show your reviewers.
4. Paste the master prompt below and send it.
5. When Kiro asks permission to run terminal commands (for example `npm install`), read the command and approve it.
6. When it finishes, open the file explorer and confirm you see `app/`, `lib/`, `middleware.ts`, `.env.example` and `README.md`.

Master prompt (copy all of it)

```
Create a complete web app called "AI Data Analytics Internship Mentor" in
this empty folder.

PURPOSE
An AI mentor chatbot that guides interns through a one-year Data Analytics
internship covering Excel, SQL, Python, Power BI, data visualization,
statistics and a final project.

TECH STACK
- Next.js (App Router) with TypeScript and Tailwind CSS
- Supabase for Auth (email + password) and Postgres, using
@supabase/supabase-js and @supabase/ssr
- Google Gemini API using the @google/genai package, called ONLY from server-
side route handlers
- Deployment target: Vercel

DATABASE (already created in Supabase, do not recreate it)
Tables: profiles(id, full_name), tasks(id, week_number, month_number, skill,
title, description, is_final_project), user_progress(id, user_id, task_id,
completed_at), chat_messages(id, user_id, role, content, created_at),
feedback(id, user_id, rating, comment, created_at).
Row Level Security is on. Each user can only access their own rows. Tasks are
readable by any logged-in user.

PAGES
/login (sign up and log in on one page), /dashboard, /roadmap, /chat,
/feedback, /completed
Add a shared top navigation bar with links to each page and a Logout button.
Every page except /login requires the user to be logged in.

ENVIRONMENT VARIABLES
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, GEMINI_API_KEY,
GEMINI_MODEL (default gemini-2.5-flash).
Create a .env.example file with these names and empty values. Make sure
.env.local is listed in .gitignore. Never hardcode secrets.

RULES
- Clean, friendly, responsive UI with an indigo/blue theme that works on
mobile
- Show loading states and readable error messages
- Keep the code simple, with short comments a beginner can follow
- Write a README.md that explains setup and how to run the project

FOR NOW
```

```
Only set up the project scaffold: install dependencies, create the Supabase client helpers (one for the browser, one for the server), middleware that protects the pages, the layout with navigation, and empty placeholder pages for each route. Do NOT build the features yet.
```

If Kiro asks a question or offers options, pick the simplest one. If anything fails, copy the full error text and paste it back into the chat with the words "Fix this error".

- [ ] Project scaffold generated
- [ ] Folder shows `app/`, `lib/`, `middleware.ts`, `.env.example`, `README.md`

Step 5: Build each feature with Kiro prompts

Build one feature at a time. After each prompt, run a quick test before moving on, so a bug is easy to find. Do Step 6 (environment variables) once before the first test, because every test needs it. To start the app, run `npm run dev` in the terminal and open `http://localhost:3000`.

Prompt 1: Login and signup

```
Build the /login page. It has two modes (tabs): Sign up (full name, email, password) and Log in (email, password), using Supabase Auth. On sign up, save the full name in the user metadata as full_name. After success, redirect to /dashboard. Show clear error messages (wrong password, email already used, weak password). The Logout button in the navbar signs out and returns to /login. If a logged-in user opens /login, redirect them to /dashboard. Keep the UI clean and centered.
```

Test: sign up with a test email, land on /dashboard, log out, log in again. In Supabase, open **Authentication, then Users** and **Table Editor, then profiles** to see your user and profile row.

Prompt 2: Roadmap data and roadmap page

```
Create supabase/seed.sql with INSERT statements for 48 rows in the existing tasks table (columns: week_number, month_number, skill, title, description, is_final_project). Use 4 weekly tasks per month, week_number 1 to 48. Monthly plan: 1 Excel basics (formulas, cleaning, sorting, filtering); 2 Excel advanced (pivot tables, charts, first dashboard); 3 SQL basics (SELECT, WHERE, GROUP BY, aggregates); 4 SQL advanced (joins, subqueries, window functions); 5 Python basics (syntax, loops, functions, files); 6 Python for analysis (pandas, NumPy); 7 Data visualization (matplotlib, seaborn, chart design); 8 Power BI basics (import, clean, reports); 9 Power BI advanced (DAX, interactive dashboards); 10 Statistics (mean, variance, distributions, hypothesis tests); 11 Statistics and storytelling (regression basics, presenting insights); 12 Final project. For each task write a short title and a 2 to 3 sentence description with one concrete deliverable, for example 'Clean a messy sales sheet and remove duplicates'. Weeks 45 to 48 are the final project, with is_final_project = true: 45 choose a dataset and define 3 business questions, 46 clean and analyze the data with SQL or Python, 47 build a dashboard, 48 write the report and present findings.

Then build the /roadmap page: read all tasks from Supabase and show them grouped by month, with the month's skill as a heading. Each task shows week number, title, description and a status badge: Done (green), Current (blue) or Upcoming (gray). The current task is the lowest week_number the logged-in user has not completed.
```

Do this yourself: open `supabase/seed.sql`, copy everything, paste it in the Supabase **SQL Editor**, and click **Run**. Check **Table Editor, then tasks** shows 48 rows.

Test: open /roadmap and confirm 12 months, with week 1 marked Current.

Prompt 3: Dashboard and progress tracking

```
Build the /dashboard page. Show: a welcome message with the user's full name; a progress bar with 'X of 48 tasks completed' and a percentage; a Current Task card with skill, week number, title, description, a 'Mark complete' button and an 'Ask the mentor about this task' button that links to /chat; and a list of the 5 most recently completed tasks with dates. 'Mark complete' inserts a row into user_progress for the current task, then refreshes the page so the next task appears. Only the current task can be completed, so interns go in order. Prevent double clicks and duplicate rows.
```

Test: click Mark complete on week 1. The bar moves to 2%, week 2 becomes current, and a row appears in `user_progress`.

Prompt 4: AI mentor chat with saved history

```
Build the /chat page and the server route POST /api/chat using the Gemini API
(@google/genai package).

API route steps: (1) verify the user is logged in with the Supabase server
client, otherwise return 401; (2) read the message from the request body; (3)
load the user's last 20 messages from chat_messages in time order; (4) find
the user's current task (lowest week_number not in user_progress); (5) call
Gemini with model process.env.GEMINI_MODEL (default gemini-2.5-flash) using
the system instruction below, the history (map role 'assistant' to 'model')
and the new message; (6) save both the user message and the assistant reply
in chat_messages; (7) return { reply }. On any error return a friendly
message and a proper status code, and never expose the API key.

System instruction text: 'You are a friendly Data Analytics Internship
Mentor. The intern is in week {week} of 48 working on: {task title} - {task
description}. Teach by guiding: give a hint first, ask the intern to try, and
give the full solution only if they ask or are still stuck after a hint.
Explain concepts in simple language with small examples. When useful, give a
short practice exercise in Excel, SQL, Python or Power BI (DAX). Keep answers
under 250 words unless asked for more. Stay on data analytics and the
internship, and politely redirect other topics.'

Chat page UI: message bubbles (user on the right, mentor on the left), render
markdown and code blocks in mentor replies, load the full saved history when
the page opens, auto-scroll to the latest message, Enter to send, a typing
indicator while waiting, disable the send button while loading, and quick
buttons: 'Give me a hint', 'Explain this task', 'Give me a practice
exercise'.
```

Test: ask "Explain GROUP BY". You should get a reply that references your current task. Refresh the page: the history must still be there. Check `chat_messages` in Supabase.

Prompt 5: Feedback system

```
Build the /feedback page. It has a 1 to 5 star rating (clickable stars) and a
comment textarea for suggestions about the learning experience. On submit,
insert a row into the feedback table with the user's id, rating and comment.
Require a rating, show a thank-you message after saving, and clear the form.
Below the form, list the user's own previous feedback with date and stars.
```

Test: submit 4 stars with a comment and confirm the row in the `feedback` table.

Prompt 6: Completion message

```
When the user has completed all tasks (the count of their user_progress rows equals the count of tasks, including the final project weeks), redirect /dashboard to /completed. Build the /completed page showing this exact message in large text: 'Congratulations! You have successfully completed your internship!' Add a simple celebration effect using CSS only, a summary (tasks completed, total months), and buttons to review the roadmap or leave feedback. If a user who has not finished opens /completed, redirect them to /dashboard.
```

Test with a shortcut: in the Supabase SQL Editor run the block below, replacing `YOUR_USER_ID` with your id from **Authentication, then Users**. Then refresh the app and the congratulations page should appear.

```
insert into public.user_progress (user_id, task_id)
select 'YOUR_USER_ID', id from public.tasks
on conflict do nothing;
```

To go back to normal testing, run `delete from public.user_progress where user_id = 'YOUR_USER_ID';`

Prompt 7: Polish and README

```
Review the whole app. Fix any TypeScript or lint errors, make sure `npm run build` passes, and make every page responsive on mobile. Add loading skeletons and empty states. Confirm no secret keys are hardcoded and that .env.local is git-ignored. Update README.md with: project description, features, tech stack, setup steps, environment variables (names only), how to run locally, and how to deploy on Vercel.
```

- [ ] Login works
- [ ] Roadmap shows 48 tasks
- [ ] Dashboard progress bar updates
- [ ] Chat answers and keeps history
- [ ] Feedback saves to the database
- [ ] Congratulations page appears at 48 of 48

Step 6: Add your keys and run the app locally

Environment variables are settings kept outside your code so secrets never reach GitHub.

1. In Kiro, find `.env.example` in the project root. Create a new file next to it named exactly `.env.local`.
2. Paste this and replace the values with your own from steps 2 and 3:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-or-publishable-key
GEMINI_API_KEY=your-gemini-key
GEMINI_MODEL=gemini-2.5-flash
```

3. Save the file. No quotes and no spaces around the `=` sign.
4. Open `.gitignore` and confirm it contains a line starting with `.env` (Next.js adds this by default). If not, ask Kiro: "Add .env.local to .gitignore".
5. In the terminal, install and start the app:

```
npm install
npm run dev
```

6. Open `http://localhost:3000` in your browser.
7. After you edit `.env.local` at any time, stop the server with `Ctrl+C` and run `npm run dev` again. Next.js only reads the file at startup.

If `gemini-2.5-flash` returns a "model not found" error, replace it with a current model name from AI Studio.

- [ ] `.env.local` created with all 4 values
- [ ] App opens at localhost:3000

Step 7: Test the full app before you upload

Run one complete pass as a brand-new intern. Reviewers will do the same, so fix anything that feels broken now.

- [ ] Sign up with a fresh email and land on the dashboard
- [ ] Roadmap shows 12 months and 48 tasks, week 1 marked Current
- [ ] Chat gives a hint-style answer that matches your current task
- [ ] Refresh the chat page: the earlier messages are still there
- [ ] Mark week 1 complete: progress bar and current task update
- [ ] Submit feedback with stars and a comment, and see it in the `feedback` table
- [ ] Create a second account: it must not see the first account's chat or progress
- [ ] Use the SQL shortcut from Prompt 6: the congratulations page shows the exact message

- [ ] Open the app in a narrow browser window or on your phone: nothing overflows
- [ ] Run `npm run build` in the terminal and confirm it ends without errors

If `npm run build` fails, paste the error into Kiro with "Fix this build error" and run it again. A build that fails locally will also fail on Vercel.

Step 8: Upload the code to GitHub

1. Go to github.com/new. Name the repository `internship-mentor-chatbot`, set it to **Public**, and leave all the "add README / .gitignore" boxes unchecked. Click **Create repository**.
2. Make sure `.env.local` is not tracked. In the Kiro terminal run `git status` right after the git add step (before git commit) and confirm `.env.local` is not listed.
3. In the Kiro terminal, inside your project folder, run these commands one by one. Replace `YOUR-USERNAME` with your GitHub username:

```
git init
git add .
git commit -m "AI Data Analytics Internship Mentor chatbot"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/internship-mentor-chatbot.git
git push -u origin main
```

4. If asked to sign in, use your GitHub account in the browser window that opens. If a password prompt appears and fails, create a **Personal access token** under GitHub **Settings, then Developer settings**, and use it as the password.
5. Refresh the repository page. You should see your files, and `.env.local` must not be among them.

For later changes, the cycle is `git add .`, `git commit -m "what changed"`, `git push`. Vercel will redeploy automatically after each push.

- [ ] Code visible on GitHub
- [ ] `.env.local` is not on GitHub
- [ ] README displays on the repository page

Step 9: Deploy on Vercel and get the shareable link

1. Go to vercel.com and log in with GitHub.
2. Click **Add New, then Project**, find `internship-mentor-chatbot`, and click **Import**.
3. Leave the framework as **Next.js** and leave the build settings as they are.

4. Open **Environment Variables** and add these four, one at a time, using the same values as your `.env.local`:

Name	Value
`NEXT_PUBLIC_SUPABASE_URL`	Your Supabase Project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Your Supabase anon / publishable key
`GEMINI_API_KEY`	Your Gemini API key
`GEMINI_MODEL`	`gemini-2.5-flash` (or the model you used locally)

5. Click **Deploy** and wait 1 to 3 minutes. When it finishes, Vercel shows a link like `https://internship-mentor-chatbot.vercel.app`. That is your shareable link.
6. In Supabase, open **Authentication, then URL Configuration**. Set **Site URL** to your Vercel link and add the same link under **Redirect URLs**. This keeps login working on the live site.
7. Open the live link in a private browser window and repeat the Step 7 checklist: sign up, chat, mark a task complete, send feedback.
8. If you forgot a variable or changed one, fix it in Vercel under **Settings, then Environment Variables**, then redeploy from the **Deployments** tab.

- [ ] Live link opens
- [ ] Signup, chat, progress and feedback work on the live site
- [ ] Supabase Site URL updated

Submission checklist and common errors

Submit these

- [ ] GitHub repository link (public, with a clear README)
- [ ] Live Vercel link
- [ ] A test account (email and password) so reviewers can log in quickly, or a note that signup is open
- [ ] 2 or 3 screenshots: dashboard, chat, congratulations page
- [ ] One paragraph describing the stack: Next.js, Supabase, Gemini, Vercel

Common errors and fixes

Problem	Likely cause	Fix
Chat says "API key not valid"	Key copied wrongly, or server not restarted	Re-copy the key with no spaces, update `.env.local`, restart `npm run dev`
Error 429 or "quota exceeded"	Gemini free-tier limit reached	Wait a minute, or ask Kiro to add a short retry and a friendly message
Error 404 "model not found"	Model name is outdated	Set `GEMINI_MODEL` to a current model name from AI Studio
"new row violates row-level security policy"	Policy missing, or `user_id` not sent	Re-run the security SQL block from Step 3, or ask Kiro to set `user_id` from the logged-in user
Roadmap page is empty	`seed.sql` was not run	Run it in the Supabase SQL Editor and check the `tasks` table has 48 rows
Login does nothing or loops	Email confirmation still on, or wrong Supabase keys	Turn off Confirm email, re-check URL and anon key
Works locally but fails on Vercel	Environment variables missing on Vercel	Add all 4 under Vercel Settings, then redeploy
Vercel build fails	TypeScript or lint error	Run `npm run build` locally, give Kiro the error, push the fix
Live login redirects to localhost	Supabase Site URL not updated	Set the Vercel link in Authentication, URL Configuration

Useful Kiro prompt when stuck

```
I got this error when I did [what you did]. Here is the full error: [paste error]. Explain the cause in simple words and fix it in the code.
```